昨天發現因為 CrossGuard 就是動態的，需要先建立 stack 才能檢查，但希望能變成類似靜態的檢查方式，或是說不要先建 stack

我在做甚麼

原本是用 pulumi preview --policy-pack 來做。研究後發現 Pulumi 的運作是：CLI 啟動 engine，engine 啟動某個東東(在 .pulumi/bin/pulumi-language-python，好像是用來把某個程式語言跑起來的)執行 program，program 執行途中每遇到一個 data source（例如 aws.ssm.get_parameter），SDK 就送一個 Invoke RPC 給 engine，engine 轉給 aws provider plugin 去打真的 AWS，program 要拿到回傳值才能繼續往下跑。

```
 aws.ssm.get_parameter(name=...)
    │
    └─ pulumi SDK 組 ResourceInvokeRequest(tok="aws:ssm/getParameter:getParameter",
...)
         │
         └─ monitor.Invoke(req)
              │
              ├─ 正常：gRPC ──► engine ──► pulumi-resource-aws ──► 真的打 AWS
              │
              └─ mock：MockMonitor.Invoke ──► Mocks.call ──► 查 INVOKE_RESULTS ──► 回傳
假值
```

我一開始想直接餵假資料給 pulumi preview，實測後確認不可能：憑證驗證那邊可以用 skipCredentialsValidation 繞過，但 data source 那邊沒辦法，沒有提供任何造假的方法。

後來找到 pulumi.runtime.set_mocks()，這是 Pulumi 官方給的單元測試 API，它同時攔截資源註冊和 data source invoke。用它就能在完全不碰 AWS 的情況下把 program 跑完、拿到它會建立的所有資源，再交給 opa rule 判斷。

取得 mock source

前面有講到，我需要一個可以用 mock data 並幫我跑完、建立 program 拿到 source 的腳本，經過這個腳本之後才能用昨天寫的 opa rule 檢查 ec2 的 imdsv2 有沒有開 (不知道為甚麼明明是檢查 imdsv2 有沒有開搞得很像在挖漏洞)，那這整個取得 source 的腳本大概做了下面的事情，其實主要是用 pulumi 的函式庫來達成的

- 找出要檢查誰，掃 repo 裡每一個帶 `Pulumi.yaml` 的目錄
- 每個 stack 開一個 subprocess 透過 `uv run --directory <stack>`，在該 stack 自己的環境裡跑，順便確保環境跟 `uv.lock` 一樣。為甚麼要開 subprocess 呢? 因為兩個 stack 在同一行程會互相污染(我在這邊看老半天看不出來到底哪裡錯...)
- 把 SDK 的線改接到自己身上，subprocess 用 `pulumi.runtime.set_mocks()`，把本來通往 engine 的 gRPC 線接到一個自己寫的物件上
- 直接把 program 跑起來，`runpy.run_path(__main__.py)`，這一行取代 `pulumi-language-python` 的工作
- 接住每一個註冊的資源，程式每建一個資源就回呼到我這邊。然後做兩件事，抄一份留著、回一個假 id 讓程式能繼續往下跑
- 我還要回答每一次 data source 查詢，程式每查一次 SSM 或 Route53 也回呼到我們手上，我們從一張查表回假值

- 然後還要等非同步註冊，裡面有一堆東西都是非同步的，所以我還要等他...
- 交給 OPA 判斷，把完整資源清單一次丟給 `opa`，用 `imdsv2.rego` 逐一檢查有沒有 EC2 資源沒開 IMDSv2
- 最後退出，通過 0、有違規 1、環境有問題跑不動回 2，pre-commit 只看這個

小結

CrossGuard 是動態的，一定要有活的 stack 跟真的 AWS 憑證才能跑，程式裡的 data source 也會真的去打 AWS。今天改成不用 preview，直接用 `pulumi.runtime.set_mocks()` 把 program 跑起來。

set_mocks 會同時攔資源註冊跟 data source 查詢。資源註冊被攔下來，程式每建一個資源我就抄一份留著，data source 被攔下來，`aws.ssm.get_parameter()` 這種拿不到回傳值就走不下去的呼叫，就可以直接回一份假資料給它。兩個都接住，program 就能完全不碰 AWS 跑完，拿到的資源清單再交給 OPA 判斷，這樣用起來就跟靜態檢查一樣，不用先建 stack。

比較意外的是兩件事，一個是兩個 stack 放在同一個行程會互相污染，只好每個 stack 開一個 subprocess，另一個是裡面很多註冊是非同步的，不等它跑完會拿到不完整的資源清單。

壞處是假資料變成我要維護的東西，每個 data source 的回傳值都寫在一張查表裡，之後新增 stack 或用到新的 data source 都得回來改這張表。

參考資料

- Unit Testing Pulumi Programs | Pulumi Docs(`set_mocks`)
- pulumi/examples testing-unit-py(Python 版 Mocks 範例)
- Pulumi AWS Provider(`skipCredentialsValidation` 等 provider 選項)
- runpy - Locating and executing Python modules
- uv run | uv Docs(`--directory`)
- Policy Language | Open Policy Agent